

ESP32 Vehicle Tracking System

Board 2 Bring-up - Daily Work Report

IoT Development Team - Car Automation Internship

1. Aim of Today's Work

The aim of today's session was to bring up the second ESP32 prototype board and test two of its main modules individually: the GPS module (for live location) and the MCP2515 CAN module (for reading vehicle parameters). The goal was to prove that each module is correctly wired and working before combining everything into the final telemetry system.

Board 2 contains three modules in total: the NEO-6M GPS, the SIM800L GSM module, and the MCP2515 CAN controller. Today we successfully tested the GPS and the CAN module. The SIM800L was started but is left for the next session.

2. Components Used

- ESP32 development board
- GY-GPS6MV2 module (u-blox NEO-6M) with ceramic patch antenna
- MCP2515 CAN module (with TJA1050 transceiver, 8 MHz crystal)
- SIM800L EVB module (started but not completed today)
- Jumper wires and USB data cable
- Arduino IDE with TinyGPSPlus and mcp_can (coryjfowler) libraries

3. Wiring / Connection Details

3.1 GPS Module (NEO-6M) to ESP32

The GPS communicates with the ESP32 using a serial (UART) connection. The TX and RX lines are crossed (GPS TX goes to ESP32 RX, and GPS RX goes to ESP32 TX).

GPS Pin	ESP32 Pin	Purpose
VCC	3V3	Power (3.3V)
GND	GND	Ground
TX	D5	GPS sends data to ESP32
RX	D4	ESP32 sends data to GPS

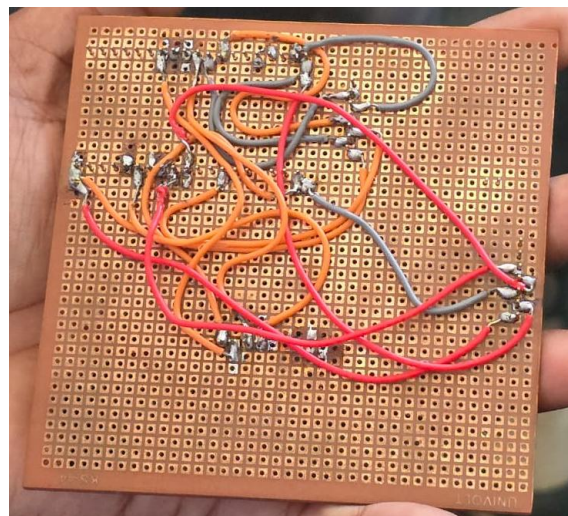
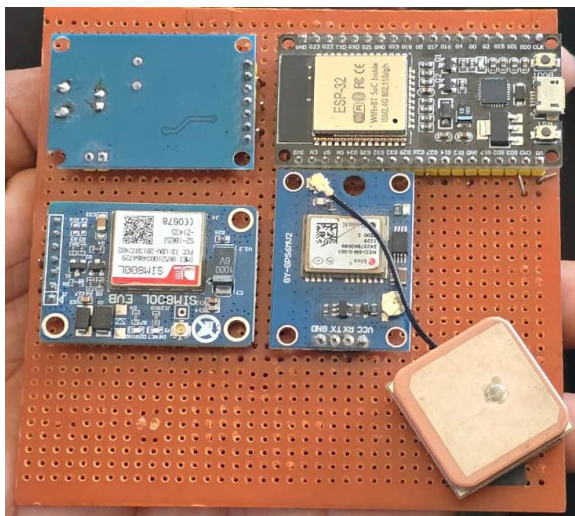
3.2 MCP2515 CAN Module to ESP32

The CAN module communicates using the SPI bus. We used the ESP32's default VSPI pins for SCK, MISO and MOSI, plus a chip-select (CS) pin and an interrupt (INT) pin.

CAN Pin	ESP32 Pin	Purpose
VCC	5V / VIN	Power (5V)
GND	GND	Ground
SCK	D18	SPI clock
SO (MISO)	D19	Data from CAN to ESP32
SI (MOSI)	D23	Data from ESP32 to CAN
CS	D32	Chip select
INT	D33	Interrupt (new message ready)

3.3 Power Summary

Connection	Supply
GPS VCC	3.3V
CAN VCC	5V / VIN
All GND	Common ground



The screenshot displays the Arduino IDE interface with the ESP32 Dev Module selected. The code in the main editor is for a file named `esp32_board2_bringup.ino`. It includes comments for OBJECTS, defines pins for GPS, serial communication, and CAN, and contains a `setup` function that initializes the serial port and prints a test message.

```
1 // ----- OBJECTS -----
2 TinyGPSPlus gps;
3 HardwareSerial GPSserial(1);
4 HardwareSerial SIMserial(2);
5 MCP_CAN CAN(CAN_CS);
6
7 bool canReady = true;
8 unsigned long lastReport = 0;
9 unsigned long lastCanTx = 0;
10
11 void setup() {
12   Serial.begin(115200);
13   delay(1000);
14   Serial.println();
15   Serial.println("=====");
16   Serial.println(" ESP32 Telemetry Board 2 - Bring-up Test");
17   Serial.println("=====");
18 }
```

The Serial Monitor at the bottom shows the output of the program, including GPS data, CAN status, and a test message. The baud rate is set to 115200.

```
16:13:52.460 -> SGPGSV,4,1,13,01,07,321,,02,29,309,,08,07,255,,10,48,059,19*7F
16:13:52.524 -> SGPGSV,4,2,13,16,04,197,16,18,07,153,26,23,28,100,31,25,19,080,25*73
16:13:52.589 -> SGPGSV,4,3,13,26,22,178,26,27,15,2(CAN) FIX lat 12.920191 lng 80.131724 sats 7 alt 33.4m spd 0.2 km/h 10:43:52 UTC
16:13:52.652 -> 24,26,28,84,012,21,31,61,215,31*74
16:13:52.684 -> SGPGSV,4,4,13,32,37,004,19*42
16:13:52.717 -> SGRCML,1255.21138,N,08007.90346,E,104352.00,A,*6D
16:13:53.224 -> SGRCMC,104353.00,A,1255.21138,N,08007.90346,E,0.025,,300626,,,*7B
16:13:53.287 -> SGPVTCG,T,M,0.025,N,0.047,K,*27
16:13:53.320 -> SGPGGA,104353.00,1255.21138,N,08007.90346,E,1,07,1.75,33.5,M,-07.5,M,*47
16:13:53.416 -> SGPGGA,A,3,18,28,31,26,27,23,25,,,,,2.56,1.75,1.07*08
16:13:53.448 -> SGPGSV,4,1,13,01,07,321,,02,29,309,,08,07,255,,10,48,059,20*75
16:13:53.545 -> SGPGSV,4,2,13,16,04,197,16,18,07,153,26,23,28,100,31,25,19,080,24*72
16:13:53.610 -> SGPGSV,4,3,13,26,22,178,26,27,15,224,26,28,84,012,21,31,61,215,31*74
16:13:53.674 -> SGPGSV,4,4,13,32,37,004,19*42
16:13:53.706 -> SGRCML,1255.21138,N,08007.90346,E,104353.00,A,*64
```

```
esp32_board2_bringup.ino
135 Serial.printf("[CAN] ID 0x%03lX len %d data:", rxId, len);
136 for (int i = 0; i < len; i++) Serial.printf(" %02X", buf[i]);
137 Serial.println();
138 }
139 }
140
141 // ----- status report every 2 s -----
142 if (millis() - lastReport > 2000) {
143   lastReport = millis();
144   if (gps.location.isvalid()) {
145     Serial.printf("[GPS] FIX lat %.6f lng %.6f sats %d alt %.1fm spd %.1f km/h %02d:%02d:%02d UTC\n",
146       gps.location.lat(), gps.location.lng(), gps.satellites.value(),
147       gps.altitude.meters(), gps.speed.kmph(),
148       gps.time.hour(), gps.time.minute(), gps.time.second());
149   } else {
150     Serial.printf("[GPS] no fix yet (NMEA chars seen: %lu, sats: %d) - give it a clear sky view\n",
151       eos.charsProcessed(), eos.satellites.value());
152   }
153 }
```

Output gdb-server Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM9') Carriage Return 115200 baud

16:17:43.458 -> [GPS] FIX lat 12.920169 lng 80.131732 sats 7 alt 34.4m spd 0.3 km/h 10:47:43 UTC
16:17:45.440 -> [GPS] FIX lat 12.920172 lng 80.131728 sats 7 alt 34.6m spd 0.7 km/h 10:47:45 UTC
16:17:47.447 -> [GPS] FIX lat 12.920172 lng 80.131729 sats 7 alt 34.5m spd 0.0 km/h 10:47:47 UTC
16:17:49.436 -> [GPS] FIX lat 12.920172 lng 80.131731 sats 7 alt 34.4m spd 0.1 km/h 10:47:49 UTC
16:17:51.432 -> [GPS] FIX lat 12.920175 lng 80.131728 sats 7 alt 34.3m spd 0.4 km/h 10:47:51 UTC
16:17:53.453 -> [GPS] FIX lat 12.920177 lng 80.131727 sats 7 alt 34.4m spd 0.6 km/h 10:47:53 UTC
16:17:55.434 -> [GPS] FIX lat 12.920175 lng 80.131729 sats 7 alt 34.4m spd 0.1 km/h 10:47:55 UTC
16:17:57.473 -> [GPS] FIX lat 12.920171 lng 80.131730 sats 7 alt 34.5m spd 0.5 km/h 10:47:57 UTC
16:17:59.451 -> [GPS] FIX lat 12.920170 lng 80.131728 sats 7 alt 34.4m spd 0.4 km/h 10:47:59 UTC
16:18:01.447 -> [GPS] FIX lat 12.920172 lng 80.131727 sats 7 alt 34.3m spd 0.3 km/h 10:48:01 UTC
16:18:03.438 -> [GPS] FIX lat 12.920177 lng 80.131725 sats 7 alt 34.2m spd 0.9 km/h 10:48:03 UTC
16:18:05.471 -> [GPS] FIX lat 12.920179 lng 80.131722 sats 7 alt 33.9m spd 0.6 km/h 10:48:05 UTC
16:18:07.442 -> [GPS] FIX lat 12.920179 lng 80.131722 sats 7 alt 33.8m spd 0.1 km/h 10:48:07 UTC
16:18:09.484 -> [GPS] FIX lat 12.920181 lng 80.131721 sats 7 alt 33.8m spd 0.2 km/h 10:48:09 UTC
16:18:11.467 -> [GPS] FIX lat 12.920178 lng 80.131723 sats 7 alt 33.7m spd 0.3 km/h 10:48:11 UTC

Ln 103, Col 1 ESP32 Dev Module on COM9

5.2 CAN Output (Loopback Self-Test)

The CAN module was tested in loopback mode. In this mode the MCP2515 sends a test message and then receives its own message back. This proves the chip and the SPI wiring work without needing a real car. Sample output:

```
[CAN] loopback test frame sent (0x123)
[CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
```

The message that was sent (DE AD BE EF 01 02 03 04) came back correctly, which means the SPI wiring, the CS pin, the INT pin and the MCP2515 chip are all working. The GPS and CAN outputs were also seen running together at the same time without any problem.

```
esp32_board2_bringup | Arduino IDE 2.3.10
Restore
Move
Size
Minimize
Maximize
Close Alt+F4
ESP32 Dev Module
pino
blue MCP2515+TJA1050 boards are 8 MHz.
CAN_CRYSTAL MCP_8MHz
45
46 // Set to true to self-test CAN on the bench with NO vehicle attached.
47 // In loopback the controller hears its own transmitted frame, which proves
48 // the SPI wiring and the chip work without needing a real bus + 120R.
49 #define CAN_LOOPBACK_TEST true
50
51 // ----- OBJECTS -----
52 TinyGPSPlus  gps;
53 HardwareSerial GPSSerial(1);
54 HardwareSerial SIMSerial(2);
55 MCP_CAN      CAN(CAN_CS);
56
57 bool canReady = true;
58 unsigned long lastReport = 0;
59 unsigned long lastCanTx = 0;
Output gdb-server Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM9')
Carriage Return 115200 baud
16:25:10.061 -> [CAN] loopback test frame sent (0x123)
16:25:10.061 -> [CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
16:25:10.061 -> [GPS] FIX lat 12.920275 lng 80.131711 sats 8 alt 32.7m spd 0.6 km/h 10:55:09 UTC
16:25:12.059 -> [GPS] FIX lat 12.920266 lng 80.131714 sats 6 alt 33.2m spd 0.1 km/h 10:55:11 UTC
16:25:12.059 -> [CAN] loopback test frame sent (0x123)
16:25:12.059 -> [CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
16:25:14.087 -> [GPS] FIX lat 12.920251 lng 80.131720 sats 6 alt 33.4m spd 0.9 km/h 10:55:13 UTC
16:25:14.087 -> [CAN] loopback test frame sent (0x123)
16:25:14.087 -> [CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
16:25:16.065 -> [CAN] loopback test frame sent (0x123)
16:25:16.065 -> [CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
16:25:16.065 -> [GPS] FIX lat 12.920242 lng 80.131723 sats 7 alt 33.4m spd 1.0 km/h 10:55:15 UTC
16:25:18.055 -> [GPS] FIX lat 12.920231 lng 80.131726 sats 7 alt 33.6m spd 0.9 km/h 10:55:17 UTC
16:25:18.086 -> [CAN] loopback test frame sent (0x123)
16:25:18.086 -> [CAN] ID 0x123 len 8 data: DE AD BE EF 01 02 03 04
Ln 49, Col 31 ESP32 Dev Module on COM9 4:25 PM 30/06/2026
35°C Mostly cloudy
```

6. Issues Faced and How We Solved Them

Issue 1: GPS was not giving any fix for a long time (sats: 0).

Solution: The GPS was indoors and could not see enough satellites. A position fix needs at least 4 satellites at the same time, and a first-time (cold) start can take around 12 minutes. We took the antenna to an open sky area, kept it flat and face-up, and did not reset the board. It then locked on.

Issue 2: The board kept resetting, which restarted the GPS search each time.

Solution: Every reset makes the GPS start its satellite download from the beginning. We stopped touching/resetting the board and let it run continuously, after which it got the fix.

Issue 3: The Serial Monitor was full of raw GPS text and we could not read the other module replies.

Solution: An extra debug line (Serial.write) was printing every raw GPS sentence. We removed that line so the program only prints the clean status messages.

Issue 4: CAN module printed nothing on the bench.

Solution: On the bench there is no real vehicle bus, so in normal mode the CAN module has nothing to receive. We switched ON the loopback self-test (CAN_LOOPBACK_TEST = true), which makes the module talk to itself, and the frames appeared.

Issue 5: SIM800L did not reply 'OK' to the AT command.

Solution: The module stayed silent. This is left for the next session - we will re-check its power supply (it needs a strong 5V), the power/PWR button on the module, the line-ending setting (Both NL & CR), and that the TX/RX wires are crossed correctly.

7. Result / Conclusion

Today both the GPS module and the MCP2515 CAN module on Board 2 were tested successfully:

- GPS - working. It gave a correct, accurate live location with 6-7 satellites.
- CAN (MCP2515) - working. The loopback self-test proved the chip and SPI wiring are correct.
- SIM800L - not completed. It did not respond and is kept for the next session.

Both working modules also ran together in the same program without any conflict, which is a good sign for combining everything later. The next steps are to fix the SIM800L and get any simple output.

